

Selecting Dependent and Independent Variables

Credit Card Approval Prediction System | Epic 4: Data Preprocessing and Labeling

Objective

Define the 17 input features (X) and binary target (y) used to train the Random Forest classifier, matching the Flask form fields exactly.

Independent Variables (X — Features)

#	Column	Type	Form Input
1	CODE_GENDER	Categorical (M/F)	Select: Male/Female
2	FLAG_OWN_CAR	Categorical (Y/N)	Select: Yes/No
3	FLAG_OWN_REALTY	Categorical (Y/N)	Select: Yes/No
4	CNT_CHILDREN	Numeric (int)	Number input (0–20)
5	AMT_INCOME_TOTAL	Numeric (float)	Number input (annual income USD)
6	NAME_INCOME_TYPE	Categorical	Select: Working/Commercial/Pensioner/etc.
7	NAME_EDUCATION_TYPE	Categorical	Select: Academic/Higher/etc.
8	NAME_FAMILY_STATUS	Categorical	Select: Married/Single/etc.
9	NAME_HOUSING_TYPE	Categorical	Select: House/Rented/etc.
10	DAYS_BIRTH	Numeric (int, negative)	Friendly: age in years → converted by JS
11	DAYS_EMPLOYED	Numeric (int, negative)	Friendly: experience in years → converted by JS
12	FLAG_MOBIL	Binary (0/1)	Hidden field, always 1
13	FLAG_WORK_PHONE	Binary (0/1)	Select: Yes/No
14	FLAG_PHONE	Binary (0/1)	Select: Yes/No
15	FLAG_EMAIL	Binary (0/1)	Select: Yes/No
16	OCCUPATION_TYPE	Categorical	Select (18 types + Unknown)
17	CNT_FAM_MEMBERS	Numeric (float)	Number input (1–25)

Dependent Variable (y — Target)

TARGET — Binary: 0 = Approved (Low Risk), 1 = Rejected (High Risk). Derived from credit_record.csv STATUS codes: any STATUS in [1,2,3,4,5] → 1.

Code: Building X and y

```

df = application_df.merge(target_df, on='ID', how='inner')
df['OCCUPATION_TYPE'] = df['OCCUPATION_TYPE'].fillna('Unknown')
df.drop_duplicates(inplace=True)
df.drop('ID', axis=1, inplace=True)

X = df.drop('TARGET', axis=1)
y = df['TARGET']

numerical_cols = X.select_dtypes(include=['int64', 'float64']).columns.tolist()
categorical_cols = X.select_dtypes(include='object').columns.tolist()

```

Preprocessing Pipeline

```

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier

preprocessor = ColumnTransformer([
    ('num', StandardScaler(), numerical_cols),
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_cols),
])

pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', RandomForestClassifier(n_estimators=100, random_state=42)),
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
pipeline.fit(X_train, y_train)

```